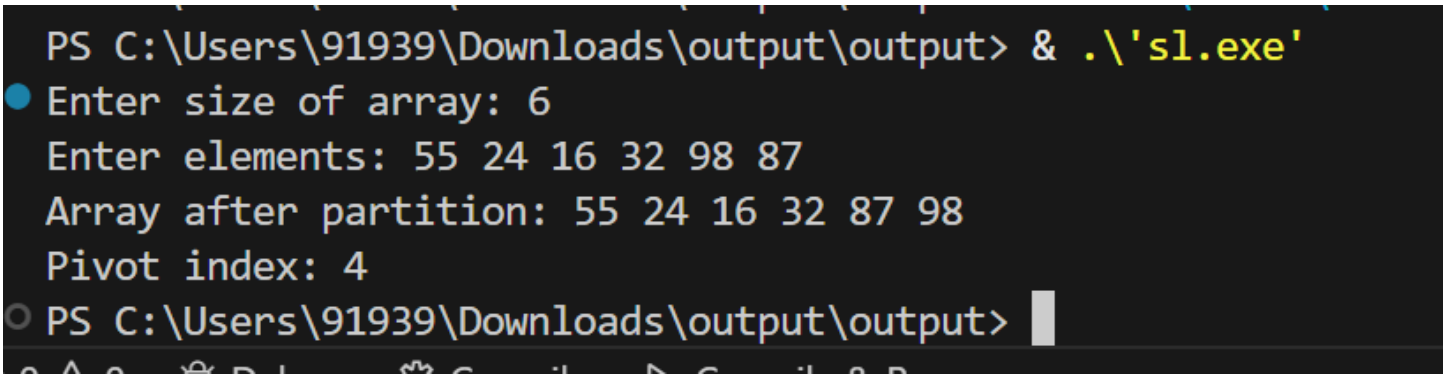
Assignment 4

Q1. Write a program to implement Partition Algorithm of Quick Sort?

```
#include <iostream>
#include <vector>
using namespace std;
int partition(vector<int>& arr, int low, int high) {
   int pivot = arr[high];
   int i = low - 1;
   for (int j = low; j < high; j++) {
      if (arr[j] <= pivot) {
         i++;
         swap(arr[i], arr[j]);
      }
   }
   swap(arr[i + 1], arr[high]);
   return i + 1;   // return pivot index
}
int main() {
   int n;
   cout << "Enter size of array: ";
   cin >> n;
       vector<int> arr(n);
   cout << "Enter elements: ";
   for (int i = 0; i < n; i++) cin >> arr[i];
   int pi = partition(arr, 0, n - 1);
   cout << "Array after partition: ";
```

```
    for (int x : arr) cout << x << " ";

    cout << "\nPivot index: " << pi << endl;

    return 0;

}
```

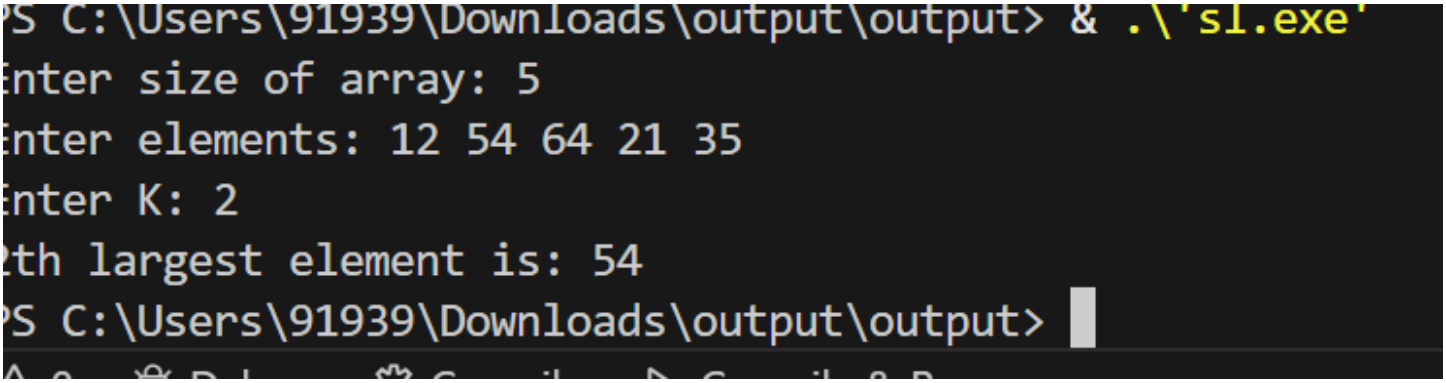

PS C:\Users\91939\Downloads\output\output> & .\'sl.exe'
Enter size of array: 6
Enter elements: 55 24 16 32 98 87
Array after partition: 55 24 16 32 87 98
Pivot index: 4
PS C:\Users\91939\Downloads\output\output>

Q2. Utilize the developed partition algorithm to identify the Kth largest number from an array of N elements. The value of K and elements should be provided by user.

```
#include <iostream>
#include <vector>
using namespace std;
int partition(vector<int> &arr, int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
    {
        if (arr[j] <= pivot)
        {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

int quickSelect(vector<int> &arr, int low, int high, int k)
{
    if (low <= high)
    {
        int pi = partition(arr, low, high);
```

```
    int n = arr.size();

    if (pi == n - k)
        return arr[pi];
    else if (pi < n - k)
        return quickSelect(arr, pi + 1, high, k);
    else
        return quickSelect(arr, low, pi - 1, k);
  }
  return -1;
}

int main()
{
  int n, k;
  cout << "Enter size of array: ";
  cin >> n;

  vector<int> arr(n);
  cout << "Enter elements: ";
  for (int i = 0; i < n; i++)
    cin >> arr[i];

  cout << "Enter K: ";
  cin >> k;

  if (k <= 0 || k > n)
  {
```

```
    cout << "Invalid K!" << endl;
    return 0;
  }

  int ans = quickSelect(arr, 0, n - 1, k);
  cout << k << "th largest element is: " << ans << endl;

  return 0;
}
```



Q2. Write a program to implement the Merge algorithm of Merge Sort?

```
#include <iostream>

#include <vector>

using namespace std;

void merge(vector<int> &arr, int l, int m, int r)

{

    int n1 = m - l + 1;

    int n2 = r - m;

    vector<int> L(n1), R(n2);

    for (int i = 0; i < n1; i++)

        L[i] = arr[l + i];

    for (int j = 0; j < n2; j++)

        R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;

    while (i < n1 && j < n2)

    {

        if (L[i] <= R[j])

            arr[k++] = L[i++];

        else

            arr[k++] = R[j++];

    }

    while (i < n1)

        arr[k++] = L[i++];
```

```
    while (j < n2)
        arr[k++] = R[j++];
}

void mergeSort(vector<int> &arr, int l, int r)
{
    if (l < r)
    {
        int m = l + (r - l) / 2;

        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);

        merge(arr, l, m, r);
    }
}

int main()
{
    int n;
    cout << "Enter size of array: ";
    cin >> n;

    vector<int> arr(n);
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    mergeSort(arr, 0, n - 1);
```

```
    cout << "Sorted array: ";

    for (int x : arr)

        cout << x << " ";

    cout << endl;

    return 0;

}
```

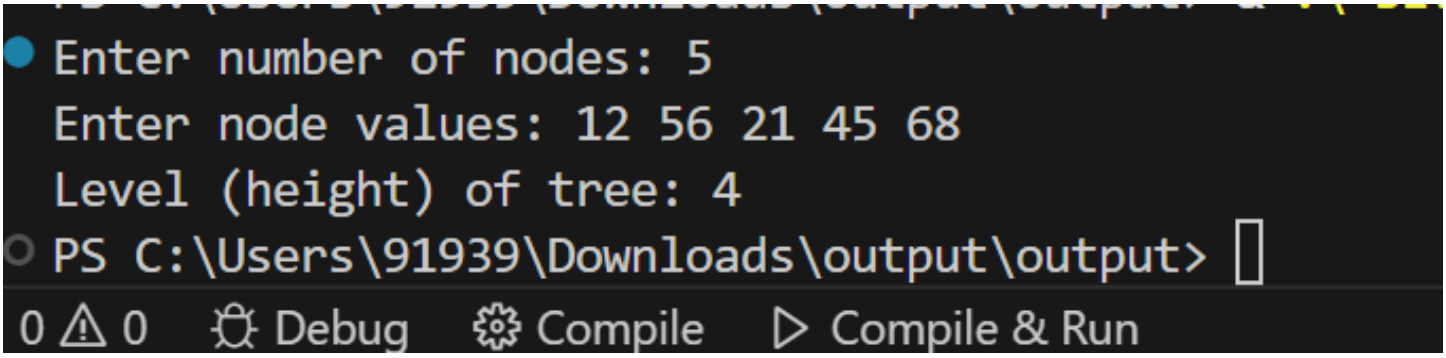
```
PS C:\Users\91939\Downloads\output\output> & .\ s1.exe
Enter size of array: 8
Enter elements: 12 54 32 161 45 231 84 33
Sorted array: 12 32 33 45 54 84 161 231
PS C:\Users\91939\Downloads\output\output>
```

Q3. Write a program to implement a binary tree and identify the level of the tree?

```
#include <iostream>

using namespace std;

// Node structure
struct Node
{
    int data;
    Node *left;
    Node *right;
    Node(int val)
    {
        data = val;
        left = right = nullptr;
    }
};

// Insert node in BST style
Node *insert(Node *root, int val)
{
    if (!root)
        return new Node(val);
    if (val < root->data)
        root->left = insert(root->left, val);
    else
        root->right = insert(root->right, val);
    return root;
```

```
}

// Function to find level/height of tree
int getLevel(Node *root)
{
    if (!root)
        return 0;
    int left = getLevel(root->left);
    int right = getLevel(root->right);
    return 1 + max(left, right);
}

int main()
{
    Node *root = nullptr;
    int n;
    cout << "Enter number of nodes: ";
    cin >> n;

    cout << "Enter node values: ";
    for (int i = 0; i < n; i++)
    {
        int val;
        cin >> val;
        root = insert(root, val);
    }

    cout << "Level (height) of tree: " << getLevel(root) << endl;
```

```
    return 0;

}
```


Enter number of nodes: 5
Enter node values: 12 56 21 45 68
Level (height) of tree: 4
PS C:\Users\91939\Downloads\output\output>
0 0
Debug
Compile
Compile & Run